

**Database:** A Database is a collection of interrelated data and a Database Management System is a software system that enables users to define, create and maintain the database and which provides controlled access to the database.

#### Database Tables:

A database most often contains one or more tables. Each table is identified by a name (e.g. "Customers" or "Orders"). Tables contain records (rows) with data. Below is an example of a table called "Persons":

| P_Id | LastName | FirstName | Address  | City      |
|------|----------|-----------|----------|-----------|
| 1    | Kumari   | Mounitha  | VPura    | Bangalore |
| 2    | Kumar    | Pranav    | Yelhanka | Bangalore |
| 3    | Gubbi    | Sharan    | Hebbal   | Tumkur    |

The table above contains three records (one for each person) and five columns (P\_Id, LastName, FirstName, Address, and City).

**SQL:** It is structured query language, basically used to pass the query to retrieve and manipulate the information from database. SQL lets you access and manipulate databases. SQL is an ANSI (American National Standards Institute) standard.

What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

#### Data Types in SQL

The following are the basic datatypes available in SQL:

##### Numeric data types:

NUMBER- used to store integer values.

NUMBER(w)- number column of specified width W.

NUMBER(w,D)-Number column of specified width out of which 2 digits are for decimal places. it's used to store floating point values

### Character-string data types:

CHAR- used to store single character.

CHAR(W)- Character column of specified width W.

VARCHAR(W) or VARCHAR2(W)- varying length character specified by W.

### A Boolean data type:

It has the traditional values of TRUE or FALSE. In SQL, because of the presence of NULL values, a three-valued logic is used, so a third possible value for a Boolean data type is UNKNOWN.

### DATE data type:

It has ten positions, and its components are YEAR, MONTH, and DAY in the form YYYY-MM-DD. The TIME data type has at least eight positions, with the components HOUR, MINUTE, and SECOND in the form HH:MM:SS. Literal values are represented by single-quoted strings preceded by the keyword DATE or TIME, for example, DATE '2008-09- 27' or TIME '09:12:47'.

### Timestamp data type:

It includes the DATE and TIME fields, plus a minimum of six positions for decimal fractions of seconds and an optional WITH TIME ZONE qualifier. For example, TIMESTAMP '2008-09-27 09:12:47.648302'.

### SQL is divided into different components:

- **DDL**(Data Definition Language )
- **DML**(Data Manipulation Language )
- **DCL**(Data Control Language )

### DDL(Data Definition Language )

The Data Definition Language (DDL) is used to create the database (i.e. tables, keys, relationships etc), maintain the structure of the database and destroy databases and database objects.

**Eg.** Create, Drop, Alter, Describe, Truncate

### Rules for column and table names

- Must begin with an alphabet.
- Cannot contain key words and spaces.
- Can contain Underscore symbol and number

### CREATE statement

It is used to create the table.

#### **Syntax:**

**CREATE TABLE** <table name>

(

<Attribute A1> <Data Type D1> [< Constarints>],

<Attribute A2> <Data Type D2> [< Constarints>],

.....

<Attribute An> <Data Type Dn> [< Constarints>],

[<integrity-constraint1>, <integrity-constraint k> ]

);

Example: create a table with name STUDENT

```
CREATE TABLE STUDENT
(
USN VARCHAR2(10)
NAME VARCHAR2(10),
DEPARTMENT CHAR(3),
SEM NUMBER(1)
);
```

A constraint NOT NULL, unique and check may be specified on an attribute as follows

```
CREATE TABLE STUDENT1
(
USN VARCHAR2(10) NOT NULL,
NAME VARCHAR2(10) NOT NULL,
DEPARTMENT CHAR(3) UNIQUE,
SEM NUMBER(1)
GENDER CHAR,
CHECK GENDER IN('M','F')
);
```

- NOT NULL Specifies that the USN COLUMN must have some value for all the rows it cannot be left blank.
- UNIQUE specifies that the values in DEPARTMENT COLUMN must be unique, no two rows can have same value for this column.
- CHECK specifies that GENDER COLUMN Can contain only M or F.

## **SQL PRIMARY KEY Constraint**

The PRIMARY KEY constraint uniquely identifies each record in a database table.

Primary keys must contain unique values.

A primary key column cannot contain NULL values.

Each table should have a primary key, and each table can have only one primary key.

Example:

```
CREATE TABLE STUDENT2
(
USN VARCHAR2(10) ,
NAME VARCHAR2(10) NOT NULL,
DEPARTMENT CHAR(3),
SEM NUMBER(1)
GENDER CHAR,
CHECK GENDER IN('M','F')
PRIMARY KEY (USN),
UNIQUE (DEPARTMENT),
CHECK GENDER IN('M','F')
);
```

OR

```
CREATE TABLE STUDENT2
(
USN VARCHAR2(10) PRIMARY KEY ,
NAME VARCHAR2(10) NOT NULL,
DEPARTMENT CHAR(3) UNIQUE,
SEM NUMBER(1)
GENDER CHAR,
CHECK GENDER IN('M','F')
CHECK GENDER IN('M','F')
);
```

- Primary key constraint ensures that the usn column cannot be blank and no two rows in student2 table can have same value for usn. Usn column values must be unique.

### **Referential Integrity Constraint**

- A constraint involving *two* relations(tables).
- Used to specify a *relationship* among tuples in two relations: the **referencing relation** and the **referenced relation**.
- A set of attribute(column) FK in referencing relation schema  $R1(\text{table})$  is a **foreign key** of  $R1(\text{table})$  that **references** relation  $R2$  if it satisfies the following rules:
  - The attributes in FK have the same domain(datatype and size) as the primary key attribute(column) PK of  $R2$ , then attributes FK are said to **reference** or **refer to** the relation  $R2$ .
  - A value of FK in a tuple(row)  $t1$  of the current state  $r1(R1)$  either occurs as a value of PK for some tuple  $t2$  in the current state  $r2(R2)$  or is *NULL*. In the former case, we have  $t1[\text{FK}] = t2[\text{PK}]$ , and we say that the tuple  $t1$  **references** or **refers to** the tuple  $t2$ .
- We can specify RESTRICT, CASCADE, SET NULL or SET DEFAULT on referential integrity constraints (foreign keys)

### **Example:**

```
CREATE TABLE EMP
(
SSN CHAR(9) PRIMARY KEY ,
NAME VARCHAR2(10) NOT NULL,
ADDRESS VARCHAR2(15) ,
CITY VARCHAR2(15),
CONTACT NUMBER(10),
SALARY NUMBER(10,2),
GENDER CHAR,
CHECK GENDER IN('M','F')
);
```

```
CREATE TABLE DEPT
(
DNAME VARCHAR(10) NOT NULL,
DNUMBER INTEGER NOT NULL,
MGRSSN CHAR(9),
MGRSTARTDATE CHAR(9),
PRIMARY KEY (DNUMBER),
UNIQUE (DNAME),
FOREIGN KEY (MGRSSN) REFERENCES EMP ON DELETE SET DEFAULT ON UPDATE CASCADE
);
```

- The values of MGRSSN column in DEPARTMENT TABLE can be only those values which are in SSN column of EMP TABLE, but more than one row can have same values.

**DESCRIBE statements:** To describe the structure (column and data types) of an existing database, table, index, or view.

**Syntax:**

DESC table\_name;

**Example:**

DESC DEPT;

**ALTER statements:**

To modify the structure of an existing table.

**Adding new columns**

**Syntax:**

```
ALTER TABLE TABLE_NAME ADD(NEW_COLUMNNAME1 DATATYPE(SIZE), NEW_COLUMNNAME2
DATATYPE(SIZE), .....NEW_COLUMNNAME2 DATATYPE(SIZE));
```

**Example:**

```
ALTER TABLE DEPT ADD(MGR_AGE INTEGER, MGR_SAL NUMBER(7,2));
```

**Dropping a columns from a table :**

**Syntax:**

```
ALTER TABLE TABLE_NAME DROP COLUMN COLUMNNAME1,.....,COLUMNNAME n;
```

**EXAMPLE:**

```
ALTER TABLE DEPT DROP COLUMN MGR_AGE;
```

A column can be dropped with the above command only if the column is empty and there are no violation of constraints like pk or fk defined on the column.

**Dropping a constraint from a table :**

**Syntax:**

```
ALTER TABLE TABLE_NAME DROP CONSTRAINTNAME;
```

**Example:**

```
ALTER TABLE DEPT DROP PRIMARY KEY;
```

Above statement drops only the constraint not the column DNUMBER from DEPT TBAL.

**Adding a constraint from a table :**

```
ALTER TABLE DEPT ADD (DNO INTEGER PRIMARY KEY);
```

**OR**

```
ALTER TABLE DEPT MODIFY (DNUMBER INTEGER PRIMARY KEY);
```

**Modifying Existing columns:**

**Syntax:**

```
ALTER TABLE TABLE_NAME MODIFY (COLUMNNAME1 NEWDATATYPE(NEWSIZE));
```

**Example**

```
ALTER TABLE DEPT MODIFY(MGR_AGE number(10), MGR_SAL NUMBER(10,2));
```

**DROP statement:**

It will drop the entire table along with the data.

**Syntax:**

```
DROP TABLE TABLE_NAME;
```

**Example**

```
DROP TABLE STUDENT;
```

**DATA MANIPULATION LANGUAGE (DML):**

A Data Manipulation Language enables programmers and users of the database to retrieve insert, delete and update data in a database. e.g. INSERT, UPDATE, DELETE, SELECT.

**INSERT statement:**

In its simplest form, it is used to add one or more tuples to a relation. Attribute values should be listed in the same order as the attributes were specified in the **CREATE TABLE command**.

**Syntax:**

```
INSERT INTO tablename VALUES (expr1,expr2..... );
```

**Example**

```
INSERT INTO DEPT VALUES ('CSE',1 , '653298653', '30-DEC-92' );
```

- Char, varchar and date type column values are enclosed within single quotes

An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple. Attributes with NULL values can be left out:

**Example:**

Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

```
INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)VALUES ('Richard', 'Marini', '653298653');
```

To leave any column blank or insert null value:

```
INSERT INTO DEPT VALUES ('CSE',1 , '653298653', ' ');
```

**MACRO TO INSERT:**

```
INSERT INTO EMPLOYEE VALUES('&FNAME','& LNAME', '&SSN');
```

### **UPDATE statement:**

UPDATE statement that changes the data of one or more records in a table. Either all the rows can be updated, or a subset may be chosen using a condition.

#### **Syntax:**

UPDATE table\_name SET column\_name1 = value ,..., column\_namen = value WHERE condition;

#### **Example1:**

UPDATE DEPT SET MGRSTARTDATE='12-jul-98';

It will update the MGRSTARTDATE to 12-jul-98 for all the rows

#### **Example2:** change the manager of department number 5

UPDATE DEPT SET MGRSSN = '653244444' WHERE DNUMBER='5';

### **SELECT Statement:**

SELECT statement returns a result set of records from one or more tables. The select statement has optional clauses:

- WHERE specifies which rows to retrieve
- GROUP BY groups rows sharing a property so that an aggregate function can be applied to each group having group.
- HAVING selects among the groups defined by the GROUP BY clause.
- ORDER BY specifies an order in which to return the rows.

#### **Syntax:**

SELECT<attribute list> FROM<table list> WHERE<condition>

#### **Where**

- Attribute list is a list of attribute name whose values to be retrieved by the query.
- Table list is a list of table name required to process query.
- Condition is a Boolean expression that identifies the tuples to be retrieved by query.

#### **Example1:** Display all the contents of table, it's called as select operation(\* indicates all)

SELECT \* FROM STUDENT1

#### **Example2:** Display contents of only single or some columns of a table, it's called as project operation

SELECT USN,NAME FROM STUDENT1;

#### **Example3:** To retrieve a subset from a table. Display the details one particular student

SELECT USN,NAME FROM STUDENT WHERE USN='4BP15CS001';

#### **Example4:** JOIN OPERATION To retrieve subset of data more than one table. Display the name of manager of department number 5(The following operation is called equijoin)

SELECT NAME FROM EMP, DEPT WHERE DEPT.MGRSSN=EMP.SSN;  
OR  
SELECT NAME FROM EMP E, DEPT D WHERE D.MGRSSN=E.SSN;

#### Example5: AND & OR OPERATOR

To retrieve the details of 5<sup>th</sup> sem CSE students.

```
SELECT * FROM STUDENT1 WHERE SEM=5 AND DEPARTMENT='CSE';
```

To retrieve the details of students in 5<sup>th</sup> sem or in CSE department.

```
SELECT * FROM STUDENT1 WHERE SEM=5 AND DEPARTMENT='CSE';
```

#### Example6: LIKE OPERATOR

To retrieve the details of students whose name starts with alphabet a.

```
SELECT * FROM STUDENT1 WHERE NAME LIKE 'a%';
```

#### Example7: DISTINCT

To retrieve the names of the city employees live.

```
SELECT DISTINCT CITY FROM EMP;
```

AGGREGATE FUNCTIONS: Used to perform aggregation on the data it Include COUNT, SUM, MAX, MIN, and AVG

Example1: Find the maximum salary, the minimum salary, and the average salary among all employees.

```
SELECT MAX (SALARY), MIN(SALARY), AVG(SALARY) FROM EMP;
```

Example2: Retrieve the total number of employees in the company.

```
SELECT COUNT (*) AS NUMBER_OF_EMPLOYESS FROM EMP;
```

#### GROUPING

- In many cases, we want to apply the aggregate functions to subgroups of tuples in a relation.
- SQL has a GROUP BY-clause for specifying the grouping attributes, which must also appear in the SELECT-clause.
- SQL has a ORDER BY-clause for sorting the data in ascending order of any specified column values by default. For sorting in descending order DSC must be explicitly at the end.

Example 1: For each department, retrieve the number of students and names of the students in alphabetical order.

```
SELECT DEPARTMENT,NAME, COUNT (USN) FROM STUDENT1 GROUP BY DEPARTMENT,NAME ORDER BY NAME;
```

#### **DELETE Statement:**

DELETE statement removes one or more records from a table. A subset may be defined for deletion using a condition, otherwise all records are removed.

#### **Syntax:**

```
DELETE FROM tablename WHERE condition;
```

Example1: To delete only the 5<sup>th</sup> sem student details from STUDENT table

```
DELETE FROM STUDENT1 WHERE SEM=5;
```

Example2: To delete the details of a student by name RAJ from STUDENT table

```
DELETE FROM STUDENT1 WHERE NAME='RAJ';
```

Example3: To delete all the data in the table

```
DELETE FROM STUDENT1;
```



### **DATA CONTROL LANGUAGE(DCL):**

These are used to control the access and maintain the database. Example: GRANT,REVOKE,COMMIT & ROLLBACK.

### **SQL TRIGGERS**

- Objective: to monitor a database and take initiate action when a condition occurs.
- Triggers are nothing but the procedures/functions that involve actions and fired/executed automatically whenever an event occurs such as an insert, delete, or update operation or pressing a button or when mouse button is clicked.

### **VIEWS IN SQL**

- A view is a single *virtual table* that is derived from other tables. The other tables could be base tables or previously defined view.
- Allows for limited update operations Since the table may not physically be stored
- Allows full query operations
- A convenience for expressing certain operations
- A view does not necessarily exist in physical form, which limits the possible update operations that can be applied to views.

### **ER- DIAGRAM:**

It is an Entity –Relationship diagram which is used to represent the relationship between different entities. An entity is an object in the real world which is distinguishable from other objects. The overall logical structure of a database can be expressed graphically by an ER diagram, which is built up from following components.

- Rectangles: represent entity sets.
- Ellipses: represent attributes.
- Diamonds: represent relationships among entity sets.
- Lines: link attribute to entity sets and entity sets to relationships.

### **MAPPING CARDINALITIES:**

It expresses the number of entities to which another entity can be associated via a relationship set. For a binary relationship set R between entity sets A and B. The Mapping Cardinalities must be one of the following.

- One to one
- One to many
- Many to one
- Many to many





To delete whole table:

delete from table\_name;

To drop a column

alter table table\_name drop (column\_name);

To change schema (add a column)

alter table table\_name add column\_name datatype;

To update:-

update table\_name

set value

where condition;

here condition is used to specify record

# 18CS53-DATABASE MANAGEMENT SYSTEM

## BASICS OF SQL

1. **Data Definition Language:** This will modify the Meta Data of the database (CREATE, ALTER, DROP instructions)

### Table Creation:

Syntax:

```
CREATE TABLE TABLE_NAME
(
  ATTRIBUTE1      DATATYPE,
  ATTRIBUTE2      DATATYPE,
  .
  .
  .
  ATTRIBUTEn      DATATYPE,
  <Constraint list>
);
```

### Example:

```
CREATE TABLE STUDENT
(
  USN VARCHAR(10),
  NAME VARCHAR(20),
  ADDRESS VARCHAR(30)
  PRIMARY KEY(USN)
);
```

The following table will be created.

| USN | NAME | ADDRESS |
|-----|------|---------|
|     |      |         |

**ALTER Instruction:** Modifying the Table Structure

**DROP Instruction:** Deleting the table

2. **Data Manipulation Language:** This will modify the Database/ Table Contents (INSERT, DELETE, UPDATE, RETRIEVE instructions)

**INSERT Instruction:** Used to insert a tuple

**Syntax:** INSERT INTO TABLENAME VALUES('Attribute1\_value','Attribute2-value',.....,'Attributen-value');

**Example:** INSERT INTO Student VALUES ('8XX19YY123', 'ABC', 'XYZ');

This will insert a new tuple as in the following table:

| USN        | NAME | ADDRESS |
|------------|------|---------|
| 8XX19YY123 | ABC  | XYZ     |

## 18CS53-DATABASE MANAGEMENT SYSTEM

**DELETE Instruction:** Used to delete one or more tuples

**Syntax:** DELETE FROM table\_name WHERE condition;

**Example:** DELETE FROM STUDENT WHERE USN='8XX19YY123';

**UPDATE Instruction:** Used to update one or more tuples

**Syntax:** UPDATE table\_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

**Example:** UPDATE STUDENT

SET NAME = 'PQR'

WHERE USN='8XX19YY123';

**RETRIEVE Instruction:** Used to fetch required data from the database without affecting the database state. These instructions are written using SELECT keyword

**Syntax:** SELECT <attribute-list>

FROM <table-list>

WHERE <condition-list>

**Example:** SELECT NAME

FROM STUDENT

WHERE USN='8XX19YY123';

**3. Data Control Language:** These instructions are used for controlling Database Operations (COMMIT, ROLLBACK, GRANT, REVOKE instructions)

**COMMIT Instruction:** Used to save the transactions executed in the database.

**ROLLBACK Instruction:** Used to revert back the changes made to the database.

**GRANT Instruction:** Used to grant permission for database user.

**REVOKE Instruction:** Used to cancel the granted permission.

### **LIKE Operator:**

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the LIKE operator:

- The percent sign (%) represents zero, one, or multiple characters.
- The underscore sign (\_) represents one, single character

## 18CS53-DATABASE MANAGEMENT SYSTEM

Here are some examples showing different **LIKE** operators with '%' and '\_' wildcards:

| LIKE Operator                  | Description                                                                  |
|--------------------------------|------------------------------------------------------------------------------|
| WHERE CustomerName LIKE 'a%'   | Finds any values that start with "a"                                         |
| WHERE CustomerName LIKE '%a'   | Finds any values that end with "a"                                           |
| WHERE CustomerName LIKE '%or%' | Finds any values that have "or" in any position                              |
| WHERE CustomerName LIKE '_r%'  | Finds any values that have "r" in the second position                        |
| WHERE CustomerName LIKE 'a_%'  | Finds any values that start with "a" and are at least 2 characters in length |
| WHERE CustomerName LIKE 'a__%' | Finds any values that start with "a" and are at least 3 characters in length |
| WHERE ContactName LIKE 'a%o'   | Finds any values that start with "a" and ends with "o"                       |

**BETWEEN Operator:** The BETWEEN operator selects values within a given range. The values can be numbers, text, or dates. The BETWEEN operator is inclusive: begin and end values are included.

**Syntax:**

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

**GROUP BY Statement:** The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country". The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns.

**ORDER BY Keyword:** The ORDER BY keyword is used to sort the result-set in ascending or descending order. The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

## 18CS53-DATABASE MANAGEMENT SYSTEM

### Query Solutions:

1. Consider the following table and apply the SQL commands for the given queries.

#### STUDENT

| <u>USN</u> | NAME | BRANCH |
|------------|------|--------|
| 123        | ABC  | IS     |
| 456        | XYZ  | CS     |
| 789        | MNO  | IS     |

- i. List the name and USN of IS Student

```
SELECT USN, NAME  
FROM STUDENT  
WHERE BRANCH='IS';
```

- ii. List the details of student with the name XYZ

```
SELECT *  
FROM STUDENT  
WHERE NAME='XYZ';
```

2. Consider the following tables and apply the SQL commands for the given queries.

#### ACTOR

| <u>ACT_ID</u> | ACT_NAME | GEN |
|---------------|----------|-----|
| A1            | ABC      | M   |
| A2            | XYZ      | F   |
| A3            | MNO      | M   |

#### DIRECTOR

| <u>DIR_ID</u> | DIR_NAME | ACT_ID |
|---------------|----------|--------|
| D1            | P        | A1     |
| D2            | Q        | A2     |
| D3            | R        | A1     |
| D4            | S        | A3     |

- i. Retrieve the names of Male Actors

```
SELECT ACT_NAME  
FROM ACTOR  
WHERE GEN='M';
```

- ii. List the names of actors who has been directed by more than one director

### 18CS53-DATABASE MANAGEMENT SYSTEM

```
SELECT A.ACT_NAME  
FROM ACTOR A, DIRECTOR D  
WHERE COUNT(D.ACT_ID)>1 AND D.ACT_ID=A.ACT_ID;
```

3. Consider the following tables and apply the SQL commands for the given queries

#### EMPLOYEE

| <u>SSN</u> | ENAME    | BDATE      | ADDRESS          | SALARY | GEN | DNUM |
|------------|----------|------------|------------------|--------|-----|------|
| E1         | GANARAJ  | 09-11-2021 | KADRI            | 10000  | M   | D1   |
| E2         | GANESH   | 10-12-2020 | ADYAR            | 20000  | M   | D2   |
| E3         | NISHANTH | 13-01-1999 | ASHOKA<br>NAGARA | 30000  | M   | D3   |
| E4         | SHRADDHA | 01-01-2001 | BANTWAL          | 40000  | F   | D3   |

#### DEPARTMENT

| <u>DNUM</u> | DNAME | MGR_SSN |
|-------------|-------|---------|
| D1          | ISE   | E1      |
| D2          | CSE   | E2      |
| D3          | ECE   | E3      |
| D4          | CIVIL | E4      |

#### PROJECT

| <u>PNUM</u> | PNAME   | P_LOC     | DNUM |
|-------------|---------|-----------|------|
| P1          | DBMS    | KASARAGOD | D1   |
| P2          | CO      | MANGALORE | D2   |
| P3          | ADE     | BC ROAD   | D3   |
| P4          | BIGDATA | PUTTUR    | D4   |

#### WORKS\_ON

| <u>ESSN</u> | <u>PNUM</u> | HOURS |
|-------------|-------------|-------|
| E1          | P1          | 10    |
| E2          | P1          | 5     |
| E3          | P2          | 5     |



## 18CS53-DATABASE MANAGEMENT SYSTEM

- i. Retrieve the Birthdate and Address of employees whose name is Ganesh

```
SELECT BDATE, ADDRESS  
FROM EMPLOYEE  
WHERE ENAME='GANESH';
```

- ii. Retrieve the name and address of all the employees working for ECE Department

```
SELECT A.ENAME, A.ADDRESS  
FROM EMPLOYEE A, DEPARTMENT B  
WHERE B.DNAME='ECE' AND B.DNUM=A.DNUM;
```

OR

```
SELECT ENAME, ADDRESS  
FROM EMPLOYEE A  
WHERE A.DNUM=(SELECT B.DNUM FROM DEPARTMENT B WHERE  
DNAME='ECE');
```

- iii. For every project located in Mangalore, list the project number, controlling department, name and address of manager of the controlling department.

```
SELECT P.PNUM, D.DNUM, E.ENAME, E.ADDRESS  
FROM EMPLOYEE E, PROJECT P, DEPARTMENT D  
WHERE P.P_LOC='MANGALORE' AND P.DNUM=D.DNUM AND  
D.MGRSSN=E.SSN;
```

- iv. Retrieve all employees name where address contains the word Nagar

```
SELECT ENAME  
FROM EMPLOYEE  
WHERE ADDRESS LIKE '%NAGAR%';
```

- v. Find all employees who were born during 1990's

```
SELECT ENAME  
FROM EMPLOYEE  
WHERE BDATE LIKE '_____ 199_';
```

## 18CS53-DATABASE MANAGEMENT SYSTEM

- vi. Show the resulting salary of every employee working on DBMS Project with 10% raise.

```
SELECT ENAME, 1.1*SALARY AS FINAL_SAL
FROM EMPLOYEE E, PROJECT P, WORKS_ON W
WHERE P.PNAME='DBMS' AND P.PNUM=W.PNUM
AND W.ESSN=E.SSN;
```

| Expected Output: ENAME | FINAL_SAL |
|------------------------|-----------|
| -----                  | -----     |
| GANARAJ                | 11000     |
| GANESH                 | 22000     |

**Note:** After analyzing the question we can understand that salary hike is not decided. So we should not update the table.

- vii. Retrieve the details of employees working in Department no.3 (D3) where salary is between 25000 and 45000

```
SELECT ENAME, SALARY
FROM EMPLOYEE
WHERE DNUM='D3' AND (SALARY BETWEEN 25000 AND 45000);
```

4. Consider the following schema and apply the SQL commands for the queries given below:

**STUDENT (USN, NAME, BRANCH, PERCENTAGE)**

**FACULTY (FID, FNAME, DEPT, DESIGNATION, SALARY)**

**COURSE (CID, CNAME, FID)**

**ENROLL (CID,USN, GRADE)**

- i. Find the names of students who have enrolled for course '18CS53'.

```
SELECT S.NAME
FROM STUDENT S, ENROLL E
WHERE E.COURSEID='18CS53' AND E.USN=S.USN;
```

- ii. Find the names of students who have enrolled for the course taught by 'Ganaraj'.

## 18CS53-DATABASE MANAGEMENT SYSTEM

```
SELECT S.NAME
```

```
FROM STUDENT S, COURSE C, FACULTY F, ENROLL E
```

```
WHERE F.FNAME='RAMESH' AND F.FID=C.FID AND C.CID=E.CID AND  
E.USN=S.USN;
```

- iii. Give a 30% hike to the salaries of ECE department faculty.

```
UPDATE FACULTY
```

```
SET SALARY=SALARY*1.3
```

```
WHERE DEPT='ECE';
```

**Note:** After analyzing the question we can understand that salary hike is decided. So we need to update the table.

- iv. List the names of students who have enrolled for the courses of 2010 scheme only.

```
SELECT S.NAME
```

```
FROM STUDENT S, ENROLL E
```

```
WHERE E.USN=S.USN AND CID LIKE '10 ____ ';
```

5. Consider the following schema and apply the SQL commands for the queries given below:

**WORKS (Person\_name, Company\_name, Salary)**

**LIVES (Person-name, Street, City)**

**LOCATED-IN (Company\_name, City)**

**MANAGER (Person\_name, Company\_name)**

- i. Find the names of all persons who live in 'ADYAR' city.

```
SELECT PERSON_NAME
```

```
FROM LIVES
```

```
WHERE CITY='ADYAR';
```

- ii. Retrieve the names of all persons of 'WIPRO' whose salary is between 10,000 and 25,000.

```
SELECT PERSON_NAME
```

## 18CS53-DATABASE MANAGEMENT SYSTEM

FROM WORKS'

WHERE (SALARY BETWEEN 10000 AND 25000) AND  
COMPANY\_NAME='WIPRO';

- iii. Find the names of persons who lives and works in the same city.

SELECT L.PERSON\_NAME  
FROM LIVES L, LOCATED\_IN M  
WHERE L.CITY=M.CITY;

- iv. List the names of people who works for 'INFOSYS' along with the cities they live in.

SELECT W.PERSON\_NAME, L.CITY  
FROM WORKS W, LIVES L  
WHERE W.PERSON\_NAME= L.PERSON\_NAME AND  
W.COMANY\_NAME='INFOSYS';

6. Consider the following schema and write the SQL queries for the following:

**EMPLOYEE (Ssn, Name, Salary, Superssn, Dno)**

**DEPARTENT (Dname, Dnum, Dmgr-ssn)**

**DEPT-LOCATIONS (Dnum, Dlocations)**

**PROJECT (PNUM, PNAME, DNUM);**

**WORKS-ON (Essn, Pno, Hours)**

**DEPENDENT (Essn, Depname, Gen)**

- i. Retrieve the number of dependents for employee Ram.

SELECT COUNT(\*)  
FROM EMPLOYEE E, DEPENDENT D  
WHERE E.NAME='RAM' AND E.SSN=D.ESSN;

- ii. Retrieve the name of managers working in location Hyderabad and has no female dependents.

SELECT E.NAME  
FROM EMPLOYEE E, DEPARTMENT D, DEPT-LOCATIONS L  
WHERE L.DLOCATIONS='HYDERABAD' AND  
L.DNUM=D.DNUM AND D.DMGR-SSN=E.SSN AND E.SSN NOT

## **18CS53-DATABASE MANAGEMENT SYSTEM**

IN (SELECT ESSN FROM DEPENDENT X WHERE E.SSN=X.ESSN  
AND X.GEN !='FEMALE');

- iii. Retrieve the name of employee who earns highest salary.

```
SELECT NAME
FROM EMPLOYEE
WHERE SALARY=MAX(SALARY);
```

- iv. Retrieve the MAX, MIN, SUM AND AVERAGE SALARY provided to the employees working in Research Department.

```
SELECT MAX(SALARY), MIN(SALARY), SUM(SALARY),
AVG(SALARY)
FROM EMPLOYEE E
WHERE DNO= (SELECT DNUM FROM DEPARTMENT WHERE
DNAME='RESEARCH);
```

- v. Find the second highest salary of an employee.

```
SELECT MAX(SALARY)
FROM EMPLOYEE
WHERE SALARY NOT IN ( SELECT MAX(SALARY) FROM
EMPLOYEE);
```

- vi. Find the names of employees who work on all the projects controlled by department number 5.

```
SELECT E.ENAME
FROM EMPLOYEE E, PROJECT P, WORK-ON W
WHERE P.DNUM=5 AND P.PNUM=W.PNO AND W.ESSN=E.SSN;
```